HOTEL MANAGEMENT SYSTEM

IT23431 –MONGODB PROJECT REPORT

Submitted by

Sabnish Seetharaman – 241001206

OF

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project titled "Secure File-Vault" is the Bonafide work of "**SABNISH SEETHARAMAN (241001206)** who carried out the project work under my supervision.

SIGNATURE	SIGNATURE
Dr.Valarmathie	Mrs.Usha S
HEAD OF THE DEPARTMENT	COURSE INCHARGE
Information Technology	Information Technology
Rajalakshmi Engineering College,	Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam	Rajalakshmi Nagar, Thandalam
Chennai – 602105	Chennai - 602105

This project is submitted for IT23431 –MongoDb Essentials held on _____

INTERNAL EXAMINAR EXTERNAL EXAMINAR

TABLE OF CONTENTS

S.No	Title	Page No
1.1	Abstract	4
1.2	Introduction	4
1.3	Purpose	5
1.4	Scope of the project	5
1.5	Software requirement specification	6
2.1	Use case diagram	9
2.2	Entity-relationship diagram	9
2.3	Data flow diagram	10
3	Module description	11
4	Implementation	13
5	Conclusion	20
6	References	20

1. Secure File Vault

1.1 ABTRACT

The Secure File Vault is a full-stack web application developed to provide users with a secure platform for storing, managing, and retrieving personal files online. The system is built using Spring Boot for backend services, MongoDB for database management, and React for frontend development. Its primary objective is to ensure data confidentiality by encrypting user files before storage and decrypting them only when authorized users request access.

The application supports essential file management features such as secure file upload, download, deletion, and user authentication. Uploaded files are converted into binary format, encrypted using secure encryption algorithms, and stored safely in the database. This process prevents unauthorized access and ensures sensitive information remains protected.

By integrating modern web technologies with cybersecurity principles, Secure File Vault minimizes the risks associated with traditional file storage systems, enhances user privacy, and demonstrates the practical implementation of secure cloud-based storage solutions. The project serves as an efficient, scalable, and reliable system for personal or organizational secure file management.

1.2 INTRODUCTION

In today's digital world, secure file storage has become a critical requirement due to increasing cybersecurity threats, unauthorized access, and data breaches. Traditional storage systems often lack adequate protection mechanisms, making sensitive user files vulnerable to theft or misuse.

The Secure File Vault addresses these challenges by offering a secure, web-based file storage solution that incorporates authentication and encryption mechanisms. Developed using Spring Boot, MongoDB, and React, the system enables users to securely upload files, which are encrypted before being stored in the database. When users request their files, the system decrypts them for safe retrieval.

This project not only simplifies digital file management but also emphasizes data security, privacy, and user accessibility. By combining secure backend processing with an intuitive user interface, Secure File Vault provides an effective solution for protecting confidential user data while ensuring convenient access anytime.

1.3 PURPOSE

The primary purpose of the Secure File Vault is to create a secure digital environment where users can safely store and manage important files without compromising confidentiality.

The system aims to:

- Protect uploaded files using encryption techniques.
- Prevent unauthorized access through user authentication.
- Provide users with convenient upload, download, and deletion capabilities.
- Ensure data integrity and confidentiality during storage and retrieval.
- Reduce security risks associated with unsecured local or cloud storage systems.
- Demonstrate practical implementation of secure full-stack web development.

Overall, the project is designed to enhance data security, improve user trust, and provide an efficient file management system suitable for modern digital needs.

1.4 SCOPE OF THE PROJECT

The Secure File Vault project focuses on providing a secure and efficient platform for users to store, manage, and retrieve digital files through a web-based application. The system is designed for individuals or organizations that require secure storage of sensitive documents, images, and other important files.

The scope of the project includes:

- Secure user registration and authentication.
- File upload functionality with encryption before storage.
- Secure file retrieval with decryption for authorized access.
- File deletion and management options.
- Storage of encrypted files in MongoDB database.
- User-friendly frontend interface for smooth interaction.
- Protection of confidential data from unauthorized users.

This project can be implemented for personal secure storage, educational purposes, or small business environments. Future enhancements may include cloud deployment, file sharing capabilities, multi-factor authentication, larger storage support, and advanced security mechanisms. The Secure File Vault provides a strong foundation for modern secure digital storage solutions.

1.5 SOFTWARE REQUIREMENT SPECIFICATION

References

- Spring Boot Documentation
- MongoDB Official Documentation
- React Documentation
- Java Cryptography Architecture (JCA)
- JWT Authentication Standards

Overall Description

The Secure File Vault system enables authorized users to securely upload, store, download, and delete files through a web application. The system ensures all uploaded files are encrypted before being stored and decrypted only when accessed by authenticated users. This improves data confidentiality, integrity, and user privacy.

Product Perspective

The application follows a client-server architecture:

- Frontend: React.js
- Backend: Spring Boot (Java)
- Database: MongoDB
- Security: JWT Authentication + File Encryption

Users interact with the React frontend, which communicates with the Spring Boot backend via REST APIs. The backend handles authentication, encryption/decryption, and database operations.

Product Functionality

a) User Registration – Allows new users to create secure accounts.
b) User Login – Provides authenticated system access using credentials.
c) File Upload – Encrypts and stores files securely.
d) File Download – Decrypts files for authorized retrieval.
e) File Deletion – Removes user files securely.
f) Dashboard – Displays user files and storage options.
g) Authentication System – Ensures secure access using JWT tokens.
h) Database Management – Handles user data and encrypted file storage.

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4GB
- Hard Disk: 20GB free storage
- Internet Connection: Required for deployment/testing
- OS: Windows/Linux/macOS

Software Requirements

- Frontend: React.js
- Backend: Spring Boot
- Database: MongoDB
- Programming Language: Java
- Build Tool: Maven
- IDE: IntelliJ IDEA / VS Code / Eclipse
- Browser: Chrome / Edge / Firefox

Assumptions and Dependencies

- Users must have valid login credentials.
- MongoDB database must be properly configured.
- Encryption keys must be securely managed.
- Backend server must remain operational.
- Internet or local server access is required.

Functional Requirements

1. Authentication Module (AM)

- User registration
- Secure login
- JWT token generation
- Session validation

2. File Management Module (FMM)

- Upload files
- Encrypt files
- Store securely
- Download files
- Decrypt files
- Delete files

3. User Dashboard Module (UDM)

- Display uploaded files
- Manage file actions
- Track user operations

4. Security Module (SM)

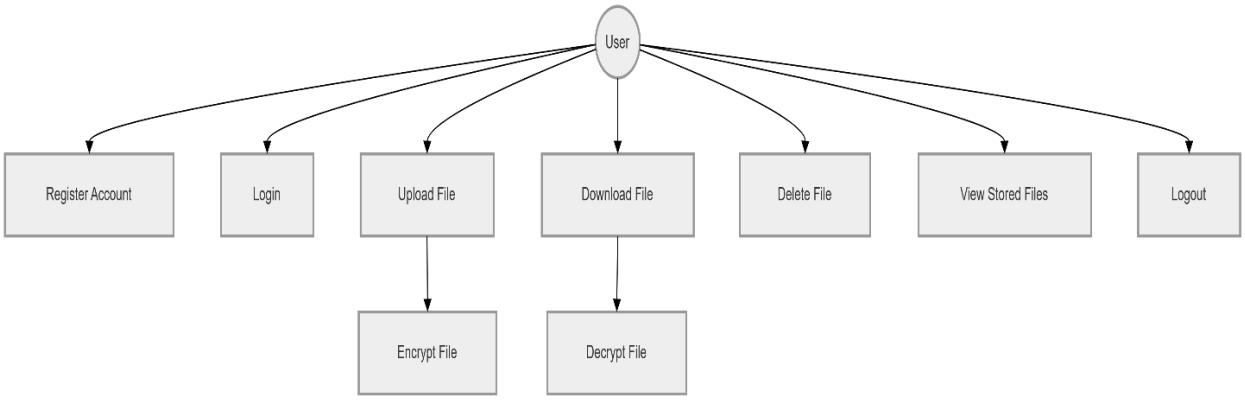
- File encryption/decryption
- Password security
- Access control
- Secure API communication

5. Database Module (DM)

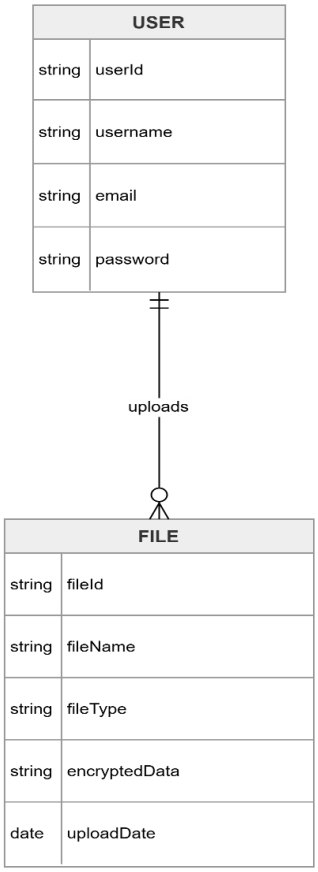
- User information storage
- File metadata storage
- Encrypted file storage
- Retrieval and deletion operations

2. SYSTEM FLOW DIAGRAMS

2.1 USE CASE DIAGRAM

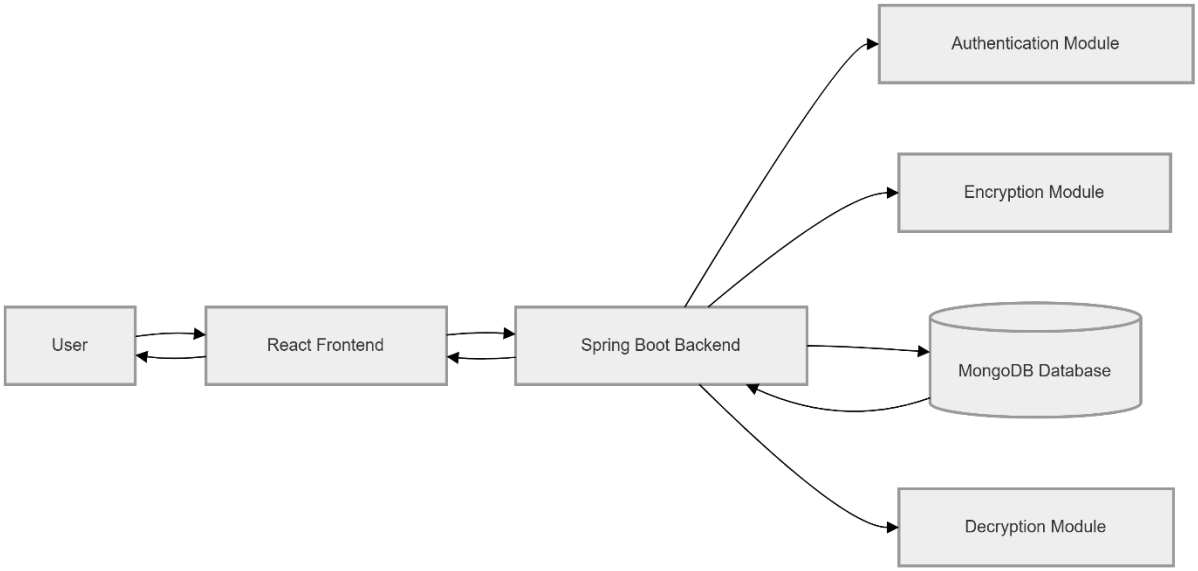


2.2 ENTITY RELATIONSHIP DIAGRAM (ER DIAGRAM)



The ER Diagram represents the relationship between users and files. Each user can upload multiple files, while each file belongs to a single user. User data includes authentication details, while file data contains encrypted file information and metadata.

2.3 DATA FLOW DIAGRAM



3. MODULE DESCRIPTION

The Secure File Vault consists of several functional modules:

USER AUTHENTICATION MODULE

- User registration
- Login validation
- JWT token generation
- Secure session management

FILE ENCRYPTION MODULE

- Encrypts files before storage
- Uses secure encryption algorithms
- Ensures confidentiality

FILE STORAGE MODULE

- Stores encrypted files in MongoDB
- Maintains file metadata
- Supports scalable storage

FILE RETRIEVAL MODULE

- Retrieves encrypted files
- Decrypts files for authorized users
- Ensures secure downloads

FILE MANAGEMENT MODULE

- View uploaded files
- Delete unwanted files
- Organize user data

FRONTEND MODULE

- User-friendly dashboard
- Responsive interface
- Secure communication with backend APIs

DATABASE MODULE

- Stores user credentials
- Stores encrypted file data
- Maintains access records

SECURITY MODULE

- Password encryption
- JWT authentication
- Role-based authorization
- Secure API access

4. IMPLEMENTATION

The implementation of the Secure File Vault system involves integrating frontend, backend, database, and security components into a fully functional web application. The project is developed using React.js for the user interface, Spring Boot for server-side processing, and MongoDB for secure data storage.

4.1 FRONTEND IMPLEMENTATION

The frontend of the Secure File Vault is designed using React.js to provide users with an interactive and responsive interface.

Key Features:

- User registration and login forms
- Secure dashboard for authenticated users
- File upload interface
- File listing and management
- Download and delete options
- API integration with backend services

Frontend Workflow:

1. User accesses the application through a web browser.
2. Registers or logs into the system.
3. Uploads files via dashboard.
4. Views stored files.
5. Downloads or deletes files securely.

Technologies Used:

- React.js
- HTML
- CSS
- JavaScript
- Axios for API communication

4.2 BACKEND IMPLEMENTATION

The backend is developed using Spring Boot, which manages authentication, file encryption, storage, and retrieval.

Backend Responsibilities:

- User authentication and authorization
- JWT token generation
- File encryption before database storage
- File decryption during retrieval
- MongoDB integration
- RESTful API services

Major Components:

- Controller Layer
- Service Layer
- Repository Layer
- Security Configuration
- Encryption Utility

4.3 DATABASE DESIGN

MongoDB is used as the NoSQL database for storing:

User Collection:

- User ID
- Username
- Email
- Password (hashed)

File Collection:

- File ID
- File Name

- File Type
- Encrypted File Data
- Upload Date
- User ID reference

Advantages:

- Flexible document storage
- High scalability
- Fast retrieval
- Secure data persistence

4.4 SECURITY IMPLEMENTATION

Security is a major component of the Secure File Vault system.

Security Features:

- JWT-based authentication
- Password hashing
- AES encryption for files
- Protected REST endpoints
- User-specific file access
- Secure login sessions

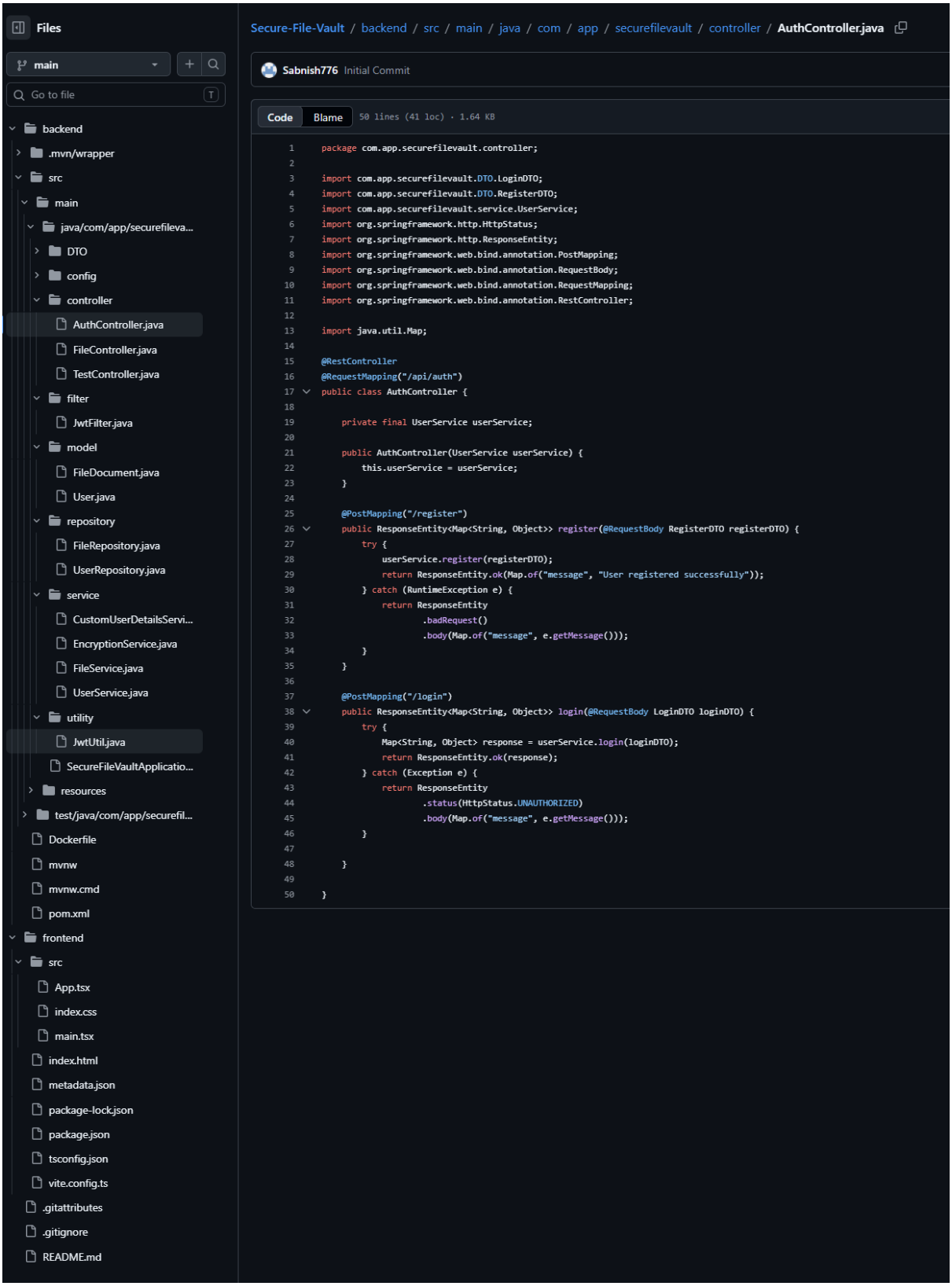
Encryption Process:

1. User uploads file
2. File converted to byte array
3. File encrypted using AES
4. Encrypted data stored in MongoDB

Decryption Process:

1. User requests file
2. Encrypted file retrieved
3. File decrypted
4. Original file returned

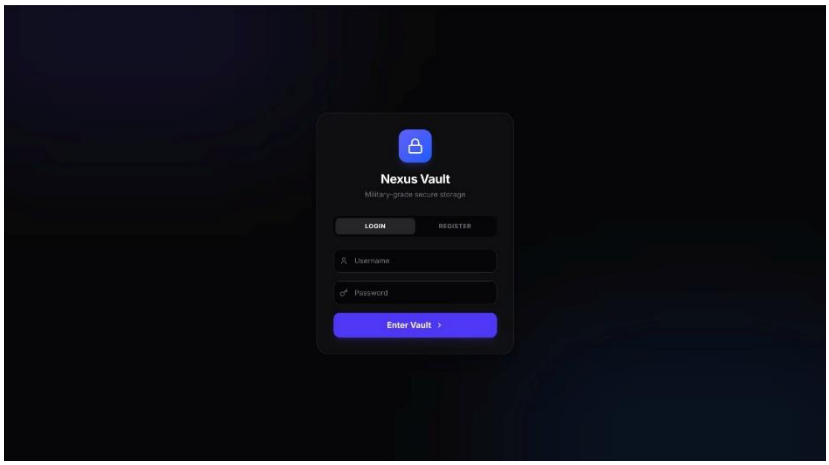
4.5 CODE STRUCTURE



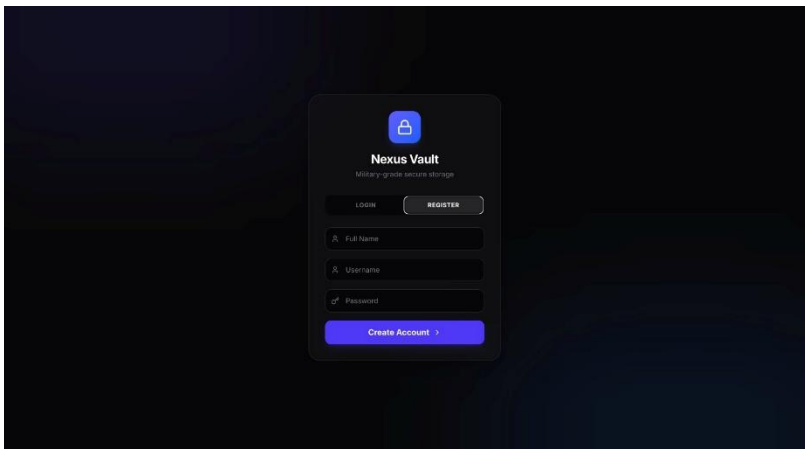
Github : https://github.com/Sabnish776/Secure-File-Vault

4.6 OUTPUT SCREENS

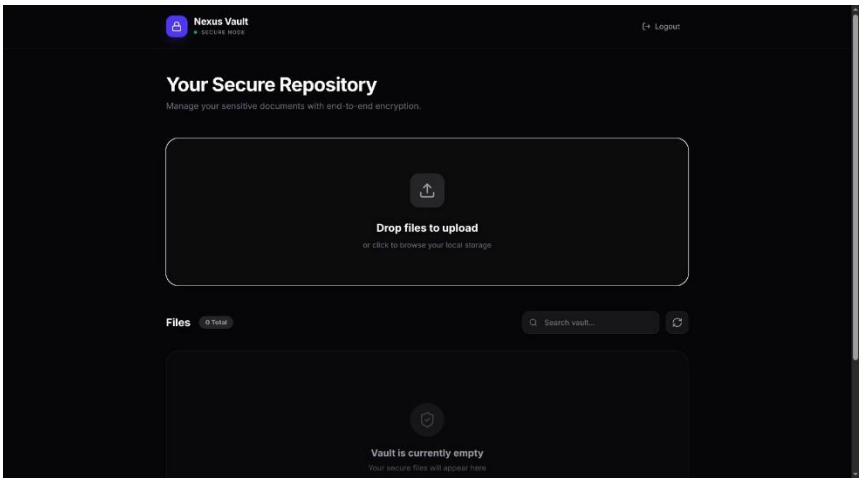
- Login Page



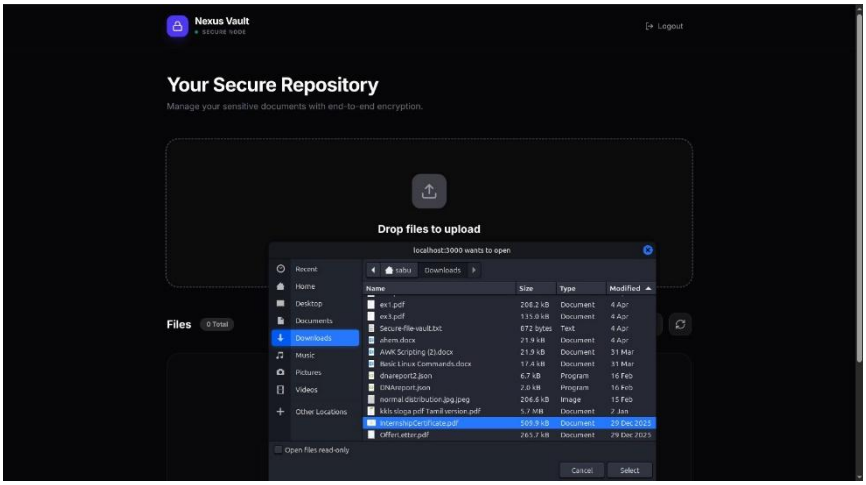
- Registration Page



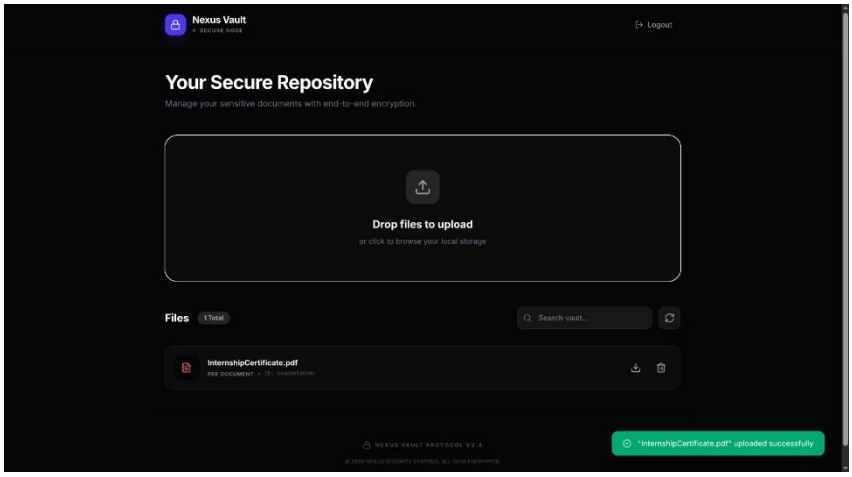
- Dashboard



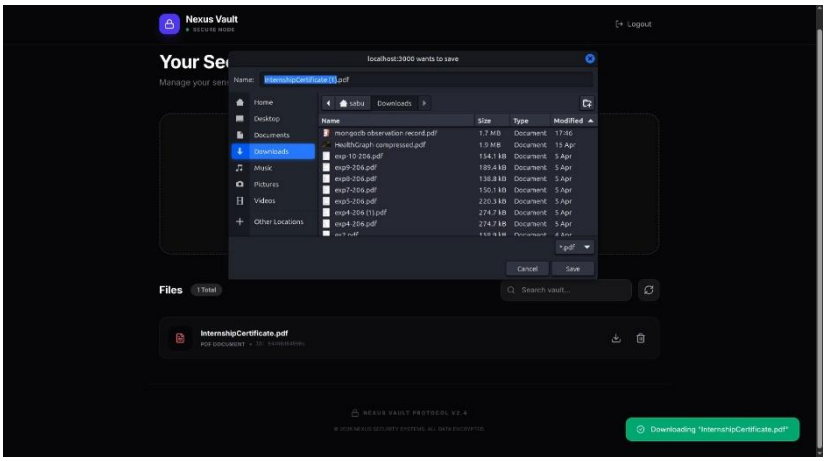
- File Upload Page



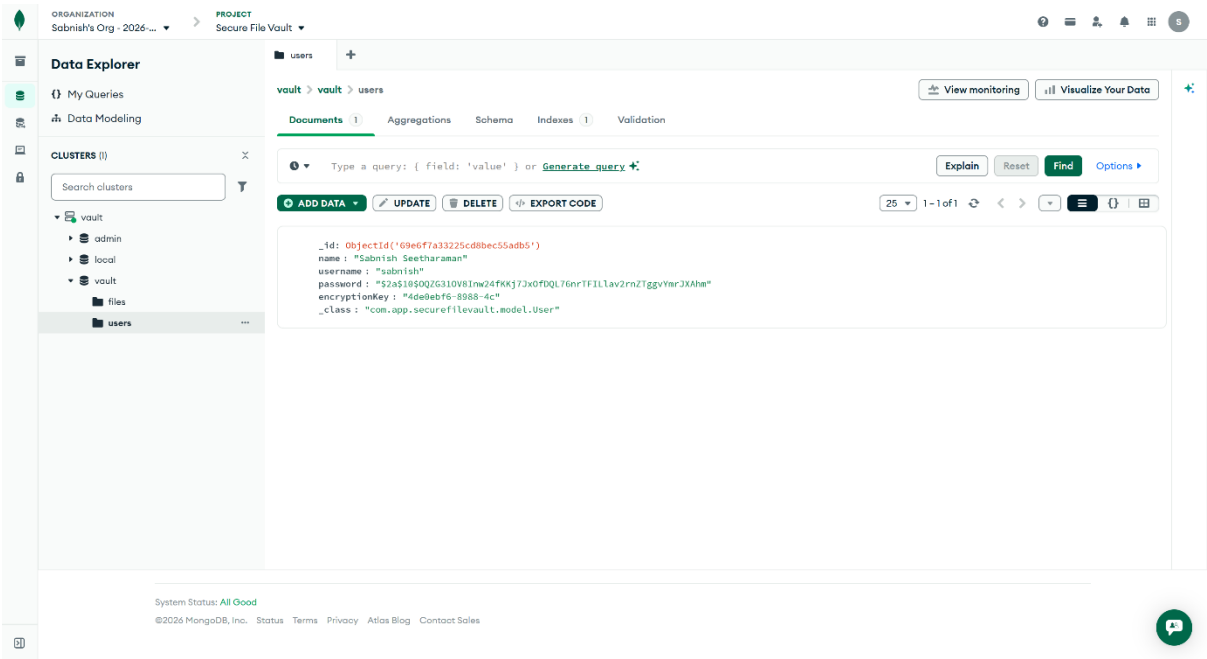
- Stored Files List

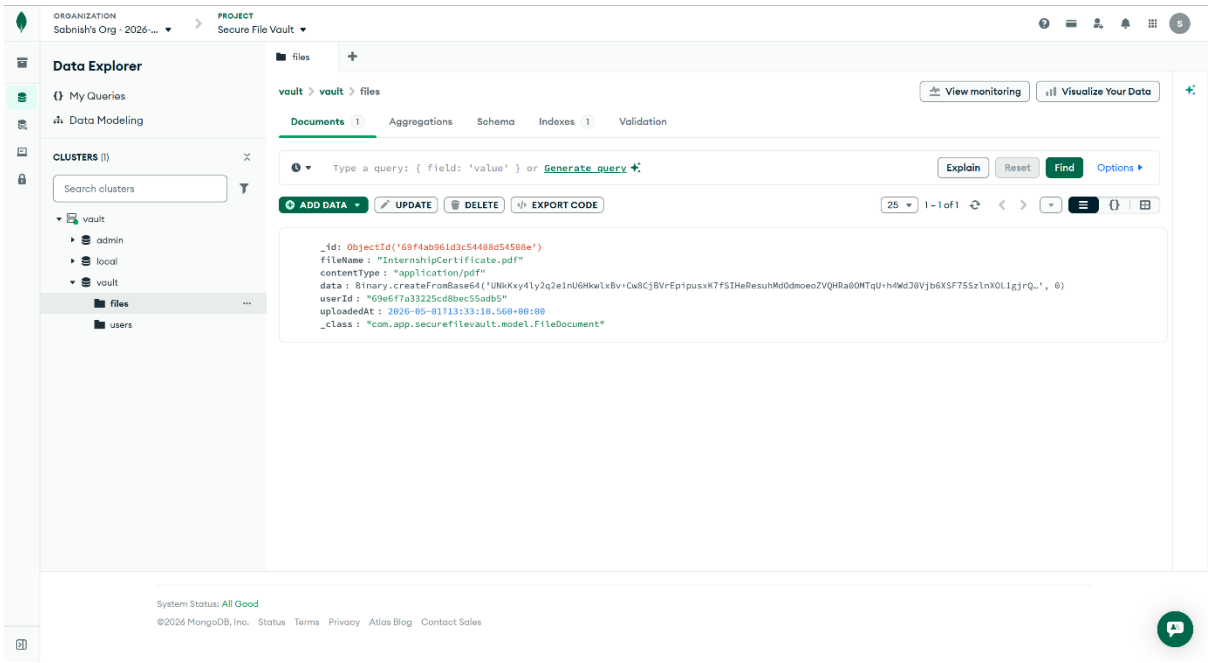


- Download Functionality



- MongoDB Collections





5. CONCLUSION

The Secure File Vault project successfully demonstrates the development of a secure and efficient web-based file storage system using modern full-stack technologies such as React.js, Spring Boot, and MongoDB. By integrating robust user authentication mechanisms and file encryption techniques, the system ensures that sensitive user files remain confidential and protected from unauthorized access.

The application provides essential functionalities including secure user registration, login, file upload, encrypted storage, file retrieval, and deletion, making it a reliable solution for digital file management. The use of encryption before storage significantly enhances data security, while JWT authentication ensures that only authorized users can access their files.

This project highlights the practical implementation of cybersecurity concepts in real-world web applications and serves as a strong foundation for future improvements such as cloud deployment, advanced sharing options, role-based access, and multi-factor authentication.

Overall, Secure File Vault is a scalable, user-friendly, and secure file management system that effectively addresses the growing need for protected digital storage solutions.

6. REFERENCES

- Spring Boot Official Documentation
- MongoDB Official Documentation
- React.js Official Documentation
- Java Cryptography Architecture (JCA) Documentation
- JWT Authentication Documentation
- GeeksforGeeks – Spring Boot Tutorials
- W3Schools – React and Java Tutorials
- Oracle Java Security Documentation
- MDN Web Docs – Frontend Development Resources